



U3 - Docker avanzado

2º DAW

2025/2026

Juanra Collado – jr.colladoesteve@edu.gva.es

RA's que se trabajan en esta unidad

- RA2 - Implanta aplicaciones web en servidores web, evaluando y aplicando criterios de configuración para su funcionamiento seguro.
 - Este RA también se trabaja en la U1, U2 y U4.
- RA3 - Implanta aplicaciones web en servidores de aplicaciones, evaluando y aplicando criterios de configuración para su funcionamiento seguro.
 - Este RA también se trabaja en la U4.

Docker compose

- Cuando necesitamos más de un contenedor, la cosa se complica.
 - Para ello nace Docker compose, que simplifica la tarea de ejecutar varios contenedores a la vez.
 - El fichero que contiene la configuración del Docker compose ha de ser llamado *docker-compose.yml* o *compose.yml* y debe incluirse en la carpeta del proyecto donde nos encontremos.
 - yml es un fichero el formato YAML, utilizado sobretodo para ficheros de configuración.
 - <https://yaml.org/>
-

Docker compose

- Docker compose nos permite definir en un solo fichero toda la estructura de nuestro aplicativo.
 - Contenedores
 - Volúmenes
 - Redes
 - De no ser así, se convertiría en algo muy difícil y farragoso de gestionar y coordinar.
 - Docker compose viene instalado por defecto en la últimas versiones de Docker.
 - De no ser así, se puede instalar aparte siguiendo las instrucciones de instalación de la página oficial <https://docs.docker.com/compose/install/>
-

Docker compose

Principales comandos de Docker Compose	
<code>docker-compose build</code>	Construye los servicios a partir de los Dockerfiles.
<code>docker-compose up</code>	Crea e inicia los contenedores del servicio.
<code>docker-compose stop</code>	Detiene un servicio, parando sus contenedores.
<code>docker-compose start</code>	Inicia un servicio parado previamente.
<code>docker-compose down</code>	Detiene y elimina un servicio, parando y eliminando los recursos asociados (contenedores, redes, imágenes, volúmenes).
<code>docker-compose ps</code>	Lista los contenedores de los servicios.
<code>docker-compose exec</code>	Ejecuta un comando en un contenedor del servicio.

Práctica 1

- Revisa que Docker compose esté instalado. Si no lo está, hazlo en este momento.
 - Puedes revisar si está instalado lanzando el comando *docker compose versión*

```
[juanra@MacBook-Air-de-Juanra Documents % docker compose version  
Docker Compose version v2.33.1-desktop.1 _
```

- Realiza el paso a paso de la guía oficial a docker compose
 - https://docs.docker.com/get-started/workshop/08_using_compose/
 - Esta práctica está resuelta en video en Aules.
-

Docker compose - Estructura

- La siguiente sección que estudiaremos es la de servicios.
- Cada servicio ha de saber a partir de qué imagen va a construir el contenedor mediante la palabra *image*.
- Podemos mapear puertos para nuestro contenedor utilizando la palabra *ports*.

Docker compose - Estructura

- Podemos setear variables de entorno mediante la palabra *environment*.

```
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"

  db:
    image: postgres:13
    environment:
      POSTGRES_USER: example
      POSTGRES_DB: exampledb
```


Docker compose - Estructura

- Podemos indicar los volúmenes del contenedor mediante *volumes*.
 - También se pueden configurar *networks*.
 - Y otras muchas opciones interesantes:
 - container_name
 - restart
 - depends_on
 - <https://docs.docker.com/compose/compose-file/>
-

¿Qué pasa con el volumen de la P1?

```
services:
  > Run Service
  app:
    image: node:lts-alpine
    command: sh -c "yarn install && yarn run dev"
    ports:
      - 3000:3000
    working_dir: /app
    volumes:
      - ./:/app
    environment:
      MYSQL_HOST: mysql
      MYSQL_USER: root
      MYSQL_PASSWORD: secret
      MYSQL_DB: todos

  > Run Service
  mysql:
    image: mysql:8.0
    volumes:
      - todo-mysql-data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: todos

  volumes:
    todo-mysql-data:
```

Docker compose - Estructura

- Cuando nos encontramos un volumen fuera de los servicios, es porque en algún momento, uno de los servicios ha declarado un *managed volume*.
- Si nos fijamos en el fichero, el servicio app declara un volumen de tipo *bind volume*, es decir, asocia directamente una carpeta local con una del contenedor

```
volumes:  
- ./:/app
```

Docker compose - Estructura

- Sin embargo, si nos fijamos en el volumen que declara el servicio *mysql*, lo que indica es el nombre de un volumen, por lo tanto nos encontramos delante de un *managed volume*.

```
volumes:  
- todo-mysql-data:/var/lib/mysql
```

- Es por ello que estamos obligados a fuera de los servicios, indicarle al fichero que va a tener que gestionar un volumen que lleve ese nombre.
 - Es decir, no es más que una declaración de un volumen que después se utiliza en un servicio.

```
volumes:  
- todo-mysql-data:/var/lib/mysql
```

```
volumes:  
todo-mysql-data:
```

Docker compose - Gestión

- Una vez tenemos finalizado nuestro fichero yml, ejecutaremos el comando para lanzar todos los servicios.
 - *docker compose up*
 - Si queremos lanzarlo en segundo plano, añadiremos -d
- Para parar los servicio ejecutaremos:
 - *docker compose down*

Práctica 2

- Crea un fichero yaml para lanzar un wordpress. Para ello necesitarás tanto un servicio de base de datos (mysql, por ejemplo) como un servicio para wordpress. Aquí algunas pistas:
 - El servicio de wordpress existe como imagen y se tiene que lanzar siempre después del servicio de base de datos.
 - Tanto el servicio de mysql como el de wordpress necesitan de variables de entorno. Lee la documentación de la imagen para saber cuales son.

Práctica 2

```
services:
  > Run Service
  db:
    image: mysql:8.0
    volumes:
      - db_data:/var/lib/mysql
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=somewordpress
      - MYSQL_DATABASE=wordpress
      - MYSQL_USER=wordpress
      - MYSQL_PASSWORD=wordpress
  > Run Service
  wordpress:
    depends_on:
      - db
    image: wordpress:latest
    ports:
      - "8000:80"
    restart: always
    environment:
      - WORDPRESS_DB_HOST=db:3306
      - WORDPRESS_DB_USER=wordpress
      - WORDPRESS_DB_PASSWORD=wordpress
      - WORDPRESS_DB_NAME=wordpress
volumes:
  db_data:
```

PAC3

- En este punto se recomienda realiza el ejercicio 1 y 2 de la PAC3.

Kubernetes

- Compose nos permite automatizar la creación de un conjunto de contenedores interconectados en un mismo equipo, lo que resulta muy cómodo para el desarrollo.
- Sin embargo, cuando una aplicación crece, o cuando queremos desplegarla en un entorno con varios servidores, Compose se queda corto.
 - Hasta 10-15 contenedor podemos utilizar docker-compose
 - Kubernetes suele utilizarse con cientos o miles de contenedores (microservicios, etc)

Kubernetes

- Aquí entra en juego **Kubernetes**, una herramienta que nació precisamente para resolver esos problemas.
 - También conocido como K8S



kubernetes

Kubernetes

- Fue creada por Google y posteriormente liberada como proyecto de código abierto
- Kubernetes es una plataforma de orquestación de contenedores.
- Se ha convertido en el estándar de facto en la industria para ejecutar aplicaciones basadas en contenedores



Kubernetes

- Compose nos permite automatizar la creación de un conjunto de contenedores interconectados en un mismo equipo, lo que resulta muy cómodo para el desarrollo.
- Sin embargo, cuando una aplicación crece, o cuando queremos desplegarla en un entorno con varios servidores, Compose se queda corto.

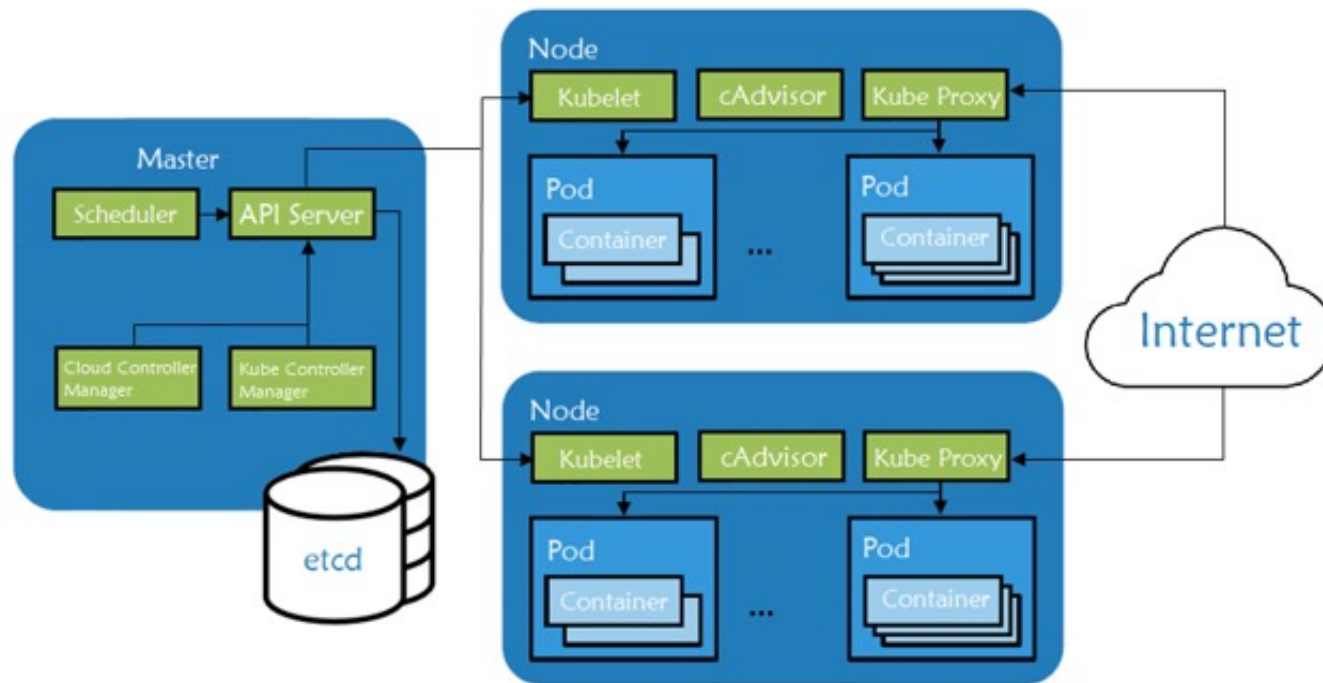
Kubernetes

- En esencia, Kubernetes se encarga de que los contenedores que forman una aplicación:
 - se desplieguen en las máquinas adecuadas,
 - se mantengan siempre en ejecución,
 - puedan replicarse para soportar más carga,
 - se actualicen sin interrumpir el servicio,
 - y se comuniquen entre sí de forma segura.
 - Kubernetes solo tiene sentido en entornos reales con varias máquinas.
 - Para fines educativos, lo haremos en local (de momento)
-

Arquitectura de Kubernetes

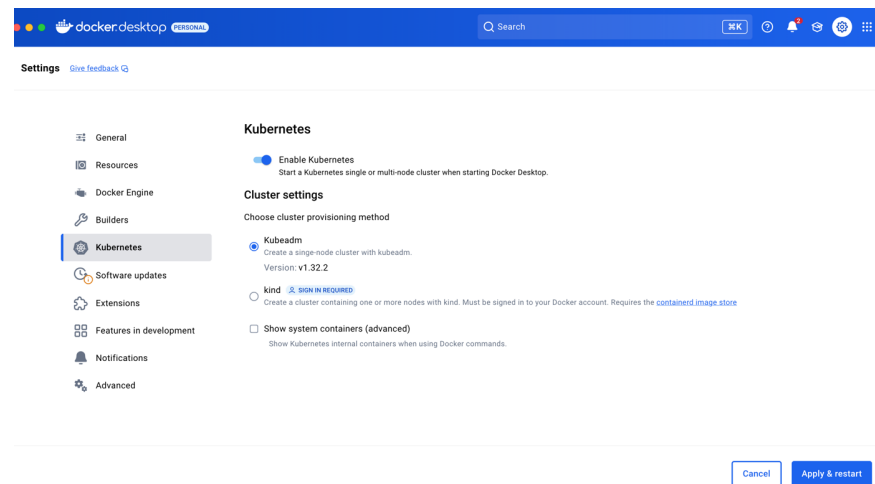
- Clúster
 - Formado por varias máquinas (físicas o virtuales)
 - Nodo maestro (control plane): gestiona el clúster, decide donde y cuando se ejecuta.
 - Nodos de trabajo (workers): ejecutan realmente los contenedores.
 - Conceptos importantes :
 - Pod: unidad mínima de despliegue. Contiene uno o varios contenedores
 - ReplicaSet: asegura que siempre haya el número deseado de pods.
 - Deployment: gestiona los ReplicaSet
 - Service: expose los pods mediante IP o puerto
 - Ingress: punto de entrada HTTP/s con balanceo de carga
 - ConfigMap y Secret: Almacena configuración y credenciales
 - Algunos más: persistenvolume, etc.
-

Arquitectura de Kubernetes



Instalación de Kubernetes

- Si estamos en un entorno con Docker Desktop:
 - Settings -> Kubernetes -> Enable Kubernetes



Instalación de Kubernetes

- Una vez arrancado, podemos comprobar que funciona correctamente ejecutando la siguiente instrucción en el terminal:

- *kubectl get nodes*

```
[juanra@MacBook-Air-de-Juanra wp % kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
docker-desktop      Ready    control-plane  2m21s  v1.32.2
```

- Para la instalación sin Docker Desktop, la recomendación es utilizar minikube (kubernetes local)
 - <https://minikube.sigs.k8s.io/>
 - Y también se puede utilizar la versión oficial, aunque está pensada para servidores:
 - <https://kubernetes.io/releases/download/>
-

Ejecución de Kubernetes

- Kubernetes, al igual que docker compose, utiliza ficheros YAML a los que llama *manifestos*.
 - Estos ficheros describen el estado deseado del sistema.
 - Es decir, no lanzamos contenedores, sino que declaramos qué contenedores queremos tener lanzados en nuestro sistema.
 - De esta forma, si el contenedor se borra manualmente (o por problemas del sistema), kubernetes lo volverá a crear para que el estado del sistema siga siendo el deseado.
-

Práctica 3 Guiada

- En un entorno real, tenemos un manifiesto por cada recurso (Deployment, Service, Secret...).
- Para una introducción más sencilla, agruparemos todos los recursos en un único fichero (separaré cada recurso con --- para que se entienda donde acaba cada recurso).
- El manifiesto puede tener el nombre que queramos ya que cuando ejecutemos Kubernetes le diremos el nombre del fichero.
 - *kubectl apply -f nombreFichero.yaml*

Práctica 3 Guiada

- El manifiesto completo (de toda la práctica) está subido en Aules.

```
apiVersion: apps/v1 #versión de la API que define el recurso
kind: Deployment #tipo de recurso que se está definiendo. En este caso, es un Deployment que gestiona la creación y actualización de Pods
metadata: #información que identifica el recurso
  name: nginx-deployment
spec: #especificaciones del Deployment. Se define el comportamiento
  replicas: 1 #número de réplicas del Pod que se desean
  selector: #etiquetas que el Deployment usará para identificar los Pods que gestiona
    matchLabels:
      app: nginx
  template: #plantilla para los Pods que se crearán
    metadata:
      labels:
        app: nginx
    spec:
      containers: #definición de los contenedores que se ejecutarán en los Pods
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80 #puerto que el contenedor expondrá. No es accesible desde fuera del Pod todavía
```

Práctica 3 Guiada

- El manifiesto anterior despliega el contenedor de nginx exponiendo el puerto 80 (sección containers).
 - Además, se le indica el nombre que tendrá y cuantas replicas (pods) queremos que tenga siempre funcionando.
 - Para desplegar la aplicación kubernetes, desde la carpeta donde se encuentra el manifiesto:
 - *kubectl apply -f nombreFichero.yaml*
-

Práctica 3 Guiada

- Y para ver los pods en ejecución (en este caso debe haber solamente 1):
 - *kubectl get pods*

```
[juanra@MacBook-Air-de-Juanra k8sBasic % kubectl apply -f basic.yaml  
deployment.apps/nginx-deployment created  
[juanra@MacBook-Air-de-Juanra k8sBasic % kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
nginx-deployment-96b9d695-gknjf    1/1     Running   0           34s
```

Práctica 3 Guiada

- El deployment ha creado los Pods, pero no los ha hecho accesibles desde fuera del clúster.
 - Para ello, es necesario utilizar un *Service*, que actúa como punto de entrada.
 - Lo añadiremos en el mismo fichero. Para nuestro caso, lo separaremos por --- para que se diferencie de lo que había antes.
 - En un entorno real, es buena práctica que esté en ficheros separados.
-

Práctica 3 - Guiada

```
  app: nginx
  spec:
    containers: #definición de los contenedores que se ejecutarán en los Pods
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80 #puerto que el contenedor expondrá. No es accesible desde fuera del Pod todavía

---

apiVersion: v1
kind: Service #exponer el Deployment al exterior
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx #selecciona los Pods a los que aplicar el servicio
  ports:
    - port: 80
      targetPort: 80
  type: LoadBalancer #podríamos utilizar NodePort o ClusterIP dependiendo del caso de uso. Para localhost mejor LoadBalancer
```


Práctica 3 Guiada

- Ahora si accedemos a localhost veremos que tenemos el servidor nginx funcionando.
- A continuación, añadiremos otro contenedor (un mysql) a nuestro clúster. Para ello, en el mismo manifesto, creamos un deployment + service para el mysql.

Práctica 3 Guiada

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: root
          ports:
            - containerPort: 3306
---
apiVersion: v1
kind: Service
metadata:
  name: mysql-service
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
  clusterIP: None #Permite que otros pods del cluster se conecten a este servicio usando su nombre (mysql-service)
```

Práctica 3 Guiada

- Ahora ya estaría nuestro clúster preparado para poder utilizar nginx+mysql desde, por ejemplo, un fichero php.
 - Ten en cuenta que nginx no trae instalado por defecto php.
 - Para poder conectarnos al contenedor del clúster (por ejemplo para crear un fichero), necesitaríamos conocer el nombre del pod:
 - `kubectl get pods`
 - La conexión a su terminal es similar a la que conocemos:
 - `kubectl exec -it nombreDelPod -- bash`
 - `kubectl exec -it web-deployment-768d75b8c4-7xzs8 -- bash`
-

Escalado y balanceo

- Hasta ahora, todo lo que hemos visto se puede hacer con Docker Compose.
 - Varios contenedores interconectados
- Kubernetes permite en tiempo real reescalar nuestra aplicación. Por ejemplo, podemos pasar de 1 pod a 3 pods.
 - *kubectl scale deployment/nginx-deployment --replicas=3*

```
juanra@MacBook-Air-de-Juanra k8sBasic % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mysql-deployment-788b7575f5-541ld   1/1     Running   0           8m37s
nginx-deployment-96b9d695-6xfgn      1/1     Running   0           3m14s
nginx-deployment-96b9d695-h9bql      1/1     Running   0           36s
nginx-deployment-96b9d695-ksqqk      1/1     Running   0           36s
```

Escalado y balanceo

- Podemos modificar la página de inicio del nginx para que nos indique desde qué POD viene.
 - Recordad que hemos de instalar PHP previamente tal y como hicimos previamente en la unidad
 - *kubectl exec -it <web-pod-name> -- bash*
 - *cd /var/www/html*
 - *echo '<?php echo "Hola desde el Pod: " . getenv("HOSTNAME");?>' > index.php*
-

Escalado y balanceo

- De esta forma cada vez que vamos accediendo veremos:
 - Hola desde el Pod: nginx-deployment - 6f7c8b9d6c-xyz1
- Y veremos que el identificador va cambiando gracias al balanceo de carga que permite kubernetes

Autorrecuperación

- Otra de las características importantes de Kubernetes es la autorreparación. Es decir, si un pod cae, automáticamente Kubernetes lo levanta para que la infraestructura siga en las mismas condiciones en la que hemos definido que ha de estar.
 - Podemos eliminar un pod utilizando
 - `kubectl delete pod nombrePod`
 - Y comprobar como kubernetes lo levanta de nuevo.
-

Autorrecuperación

- Otra de las características importantes de Kubernetes es la autorreparación. Es decir, si un pod cae, automáticamente Kubernetes lo levanta para que la infraestructura siga en las mismas condiciones en la que hemos definido que ha de estar.
 - Podemos eliminar un pod utilizando
 - `kubectl delete pod nombrePod`
 - Y comprobar como kubernetes lo levanta de nuevo.
-

+ Ventajas Kubernetes

- Otra de las ventajas de Kubernetes son las actualizaciones en caliente
 - Esto significa que podemos actualizar la versión de nuestro servidor o BBDD o aplicar cualquier tipo de parche sin necesidad de pararlo.
 - Para ello, Kubernetes va reemplazando los pods antiguos gradualmente por pods nuevos, siempre manteniendo el servicio.
 - *kubectl set image deployment/nginx-deployment nginx=nginx:1.27*
 - *kubectl rollout status deployment/nginx-deployment*
-

Eliminación de deployment

- Para eliminar por completo un deployment (incluye el deployment y los services lanzados), ejecutaremos la siguiente instrucción:
 - `kubectl delete -f deployment.yaml`

```
[juanra@MacBook-Air-de-Juanra k8sBasic % kubectl delete -f basic.yaml  
  
deployment.apps "nginx-deployment" deleted  
service "nginx-service" deleted  
deployment.apps "mysql-deployment" deleted  
service "mysql-service" deleted  
[juanra@MacBook-Air-de-Juanra k8sBasic % kubectl get pods  
No resources found in default namespace._
```

¿Dudas?

